

H-FARM INTERNATIONAL SCHOOL

HIS Management Platform — Guida completa

H-FARM International School · Management Platform

Guida tecnica e operativa per chi gestisce la piattaforma

Versione del 13 ottobre 2026 — aggiornata alla migrazione database **0022**

Documento interno · non contiene password né chiavi

Indice

Come leggere questa guida

Glossario essenziale

1. Cos'è la piattaforma

2. Lo schema generale

3. Dove si trova ogni cosa

4. Le piattaforme esterne usate

5. La struttura dei file

6. Come funziona il frontend

7. Ruoli, permessi e società

8. Il backend: Supabase

9. Cosa salva il database

10. Le Edge Functions

11. Secret e chiavi: dove sono e a cosa servono

12. Come funzionano i flussi principali

13. Le migrazioni del database

14. Come pubblicare una modifica

15. Manutenzione e operazioni frequenti

16. Risoluzione dei problemi

17. Note finali

Come leggere questa guida

Questa guida spiega **com'è fatta la piattaforma, dove si trova ogni pezzo e come si gestisce**. È scritta per essere capita anche da chi non è sviluppatore: ogni termine tecnico è spiegato la prima volta che compare e c'è un glossario qui sotto.

- Se ti serve **una visione d'insieme**, leggi i capitoli 1, 2 e 3.
- Se devi **fare una modifica o pubblicarla**, vai ai capitoli 13 e 14.
- Se qualcosa **non funziona**, vai al capitolo 16.
- Se cerchi **una password, una chiave o un URL segreto**, il capitolo 11 dice dove si trova (la guida non contiene i valori, solo dove sono custoditi).

> **Importante:** questa guida **non contiene password né chiavi segrete**. Dice solo *dove* sono salvate e *a cosa servono*. Non aggiungere mai valori segreti in questo documento.

Glossario essenziale

Termine	Cosa vuol dire, in parole semplici
Frontend	La parte che vedi nel browser: pagine, pulsanti, moduli.
Backend	La parte "dietro le quinte" che salva i dati, controlla chi può fare cosa e invia le notifiche. Qui è **Supabase**.
Database	L'archivio dove sono salvati i dati, organizzato in **tabelle** (come fogli Excel con colonne fisse). Qui è un database **PostgreSQL** dentro Supabase.
Repository (repo)	La cartella del codice sorgente, con tutta la sua storia delle modifiche. Sta su **GitHub**.
Git / commit / push	Git è lo strumento che registra le modifiche al codice. *Commit* = salvare un gruppo di modifiche con una descrizione. *Push* = inviarle a GitHub.
Build	La "compilazione": trasforma il codice sorgente in pagine web pronte da pubblicare.
Deploy	La pubblicazione online della nuova versione.
Hosting	Il servizio che "ospita" il sito e lo rende raggiungibile da internet. Qui è **Cloudflare Pages**.
Edge Function	Un piccolo programma che gira sui server di Supabase (non nel browser). Serve per le operazioni che richiedono segreti, ad esempio inviare dati a Power Automate.
Secret	Un valore riservato (password, chiave, URL privato) salvato in modo cifrato nel backend, mai nel codice.
Migrazione (SQL)	Un file `.sql` che modifica la struttura del database (crea tabelle, regole, funzioni). Si esegue a mano nel SQL Editor di Supabase.
RLS (Row Level Security)	Le regole del database che decidono, riga per riga, chi può leggere o modificare i dati. Anche se qualcuno aggirasse le pagine, il database rifiuta ciò che non è permesso.
RPC / funzione SQL	Una funzione salvata nel database che la piattaforma chiama per operazioni delicate (es. cambiare un ruolo): controlla i permessi prima di agire.
Webhook	Un indirizzo web (URL) a cui un sistema invia dati per far partire un'automazione. Qui: i flow di **Power Automate**.

Termine	Cosa vuol dire, in parole semplici
MFA	Autenticazione a due fattori: oltre alla password serve il codice a 6 cifre di un'app (Microsoft Authenticator, Google Authenticator...).
JWT / token	La "tessera" digitale che il browser riceve dopo il login e che presenta al backend a ogni richiesta.
Anon key	Chiave pubblica di Supabase usata dal sito. **Non è un segreto**: la sicurezza è garantita dalle regole RLS.
Service role key	Chiave "amministratore" di Supabase. **È segretissima**: non va mai nel sito né nel codice.

1. Cos'è la piattaforma

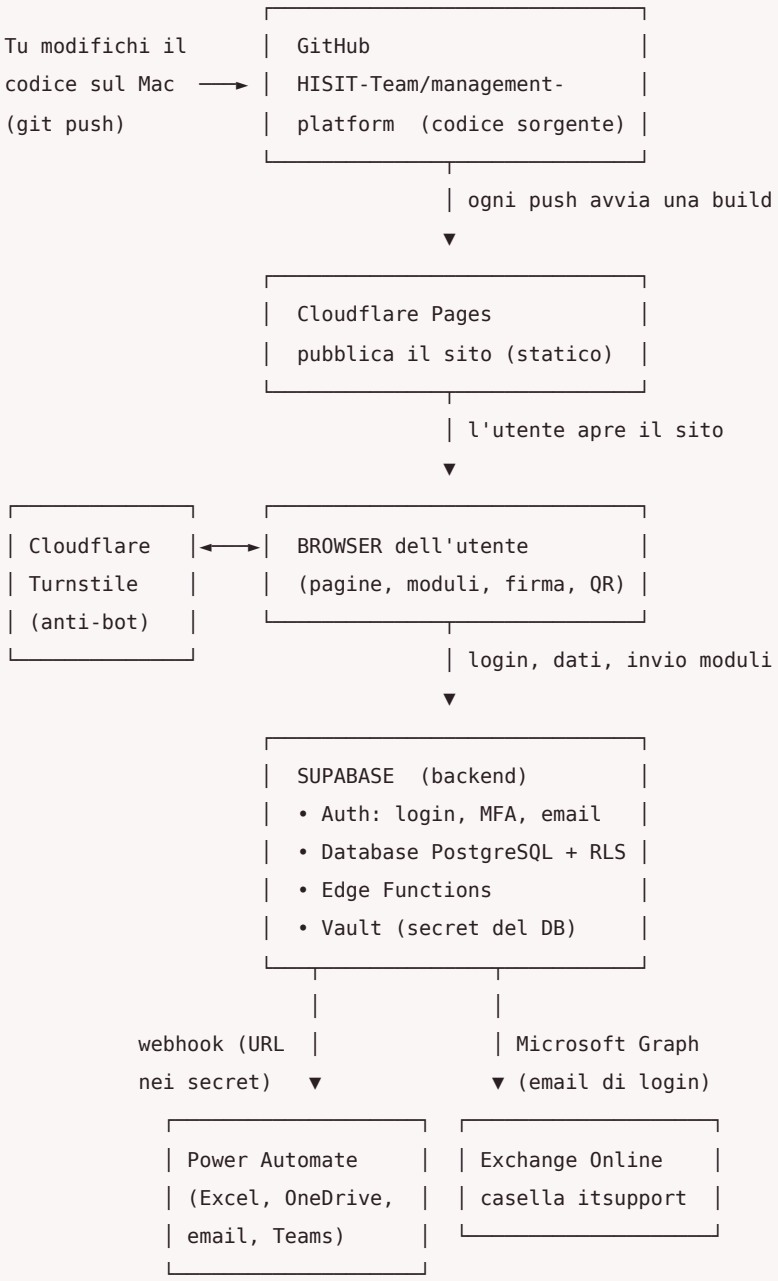
La **HIS Management Platform** è il portale interno di H-FARM International School. Ogni dipendente entra con email e password (più MFA se il suo ruolo lo richiede) e vede **solo le sezioni del proprio ruolo**.

Funzioni principali oggi attive:

Area	Cosa si fa
Dashboard	Pagina iniziale personalizzata: indicatori, ultimi movimenti, budget, scadenze.
IT	Consegna e restituzione dispositivi a studenti e dipendenti (con firma, foto, scansione QR), storico assegnazioni, registri, card QR studenti, link all'Asset Manager.
Budget	Commesse e spese delle tre scuole (Venezia, Vicenza, Rosà), con creazione commesse e modifica degli stanziamenti. *Solo Owner, Super Admin, Admin.*
Task Manager	Attività del team IT con gruppi di progetto, sotto-task, vista lista e kanban, notifiche Teams. *Solo Owner, Super Admin, Admin.*
HR	Moduli di onboarding (nuovo dipendente) e offboarding (uscita), registri.
Boarding	Assegnazione camere, link alle telecamere.
Student Office	Area per l'ufficio studenti (moduli in arrivo).
Amministrazione	Utenti e ruoli, permessi per ruolo, registro attività, sicurezza (MFA, timeout, dispositivi ricordati). *Solo Owner e Super Admin.*
Profilo	Dati personali, foto, cambio email/password, MFA, preferenze di aspetto.

La piattaforma è divisa in **tre società**: **H-IS Venezia**, **H-IS Vicenza**, **H-IS Rosà** (vedi capitolo 7).

2. Lo schema generale



In breve:

1. **Il codice** vive su GitHub.
2. **Cloudflare Pages** prende il codice da GitHub, lo "compila" e pubblica il sito.
3. **Il sito** gira nel browser e parla solo con **Supabase**.
4. **Supabase** controlla chi sei, salva i dati e, tramite le **Edge Functions**, inoltra i moduli a **Power Automate** (che scrive su Excel/OneDrive, manda email e messaggi Teams).

Il sito **non ha un server proprio**: è un insieme di pagine statiche. Tutta la logica "riservata" sta in Supabase.

3. Dove si trova ogni cosa

Cosa	Dove	Note
Codice sorgente (ufficiale)	GitHub → organizzazione **HISIT-Team**, repository **management-platform** (`https://github.com/HISIT-Team/management-platform`)	È la versione "vera". Tutto parte da qui.
Copia di lavoro sul Mac	`~/Downloads/management-platform/management-platform`	Da qui si fanno le modifiche e il `git push` .
Sito pubblicato	**Cloudflare Pages**, collegato al repository GitHub	Si aggiorna da solo a ogni push sul ramo principale. L'indirizzo pubblico è quello configurato nel progetto Cloudflare Pages (dominio `*.pages.dev` o dominio personalizzato).
Backend (Supabase)	Supabase, organizzazione **it.team@h-farmschool.com's Org**, piano **Free**. Progetto con indirizzo `https://gnahighyfoirnadhihez.supabase.co`	Dashboard: `https://supabase.com/dashboard` → progetto.
Database	Dentro Supabase → *Table Editor* / *SQL Editor*	Tabelle elencate al capitolo 9.
Edge Functions	Supabase → *Edge Functions* (il codice è anche in `supabase/functions/` nel repo)	`submit-form` , `notify-powerautomate` , `send-auth-email` .
Secret delle Edge Functions	Supabase → *Edge Functions* → *Secrets*	Elenco al capitolo 11.
Secret del database	Supabase → *Vault* (o `vault.secrets` via SQL)	URL del flow Teams e indirizzo della piattaforma.
Migrazioni SQL	`supabase/migrations/` nel repo (0001 → 0022)	Si eseguono a mano nel SQL Editor.
Flussi di automazione	**Microsoft Power Automate** (tenant H-FARM School)	Ricevono i moduli tramite webhook.
Documentazione dei flussi	`supabase/functions/notify-powerautomate/README.md` , `supabase/notifications/teams-task-assigned/`	Schema dei dati e istruzioni.
Registri Excel / foto	SharePoint / OneDrive (link nelle pagine HR e Boarding)	Scritti dai flow Power Automate.
Inventario dispositivi	Asset Manager `https://assetmanager.h-farm.com/`	Le etichette QR sui dispositivi puntano qui.
Telecamere	Cisco Meraki Vision `https://vision.meraki.com/`	Link dalla sezione Boarding.

4. Le piattaforme esterne usate

Servizio	A cosa serve	Dove si gestisce
GitHub	Custodisce il codice sorgente e la cronologia delle modifiche.	github.com → HISIT-Team/management-platform
Cloudflare Pages	Compila e pubblica il sito; applica le intestazioni di sicurezza (file `public/_headers`).	Dashboard Cloudflare → Workers & Pages → progetto della piattaforma
Cloudflare Turnstile	"Captcha" invisibile sulle pagine di login, registrazione e recupero password: blocca i bot.	Dashboard Cloudflare → Turnstile. La *site key* è nel sito, la *secret key* è in Supabase (Auth → Attack Protection).
Supabase	Il backend completo: autenticazione, database, Edge Functions, Vault.	supabase.com/dashboard
Supabase Auth	Login email+password, registrazione, recupero password, MFA (TOTP), sessioni.	Supabase → Authentication
PostgreSQL (in Supabase)	Il database con tabelle, regole RLS, funzioni e trigger.	Supabase → Table Editor / SQL Editor
pg_net (estensione)	Permette al database di chiamare un webhook (notifica Teams quando si assegna una task).	Supabase → Database → Extensions
Supabase Vault	Cassaforte cifrata per i segreti usati dal database.	Supabase → Vault
pg_cron (opzionale)	Pianificazione di lavori periodici (es. cancellare le firme vecchie). Non ancora attivo.	Supabase → Database → Extensions
Microsoft Power Automate	Riceve i moduli (consegne, onboarding, offboarding, camere) e le notifiche task; scrive su Excel, salva foto su OneDrive, invia email di riepilogo e messaggi Teams.	make.powerautomate.com
Microsoft Teams	Riceve le notifiche "ti è stata assegnata una task".	Tramite Power Automate (Flow bot)
Microsoft Entra ID + Microsoft Graph	Invio delle email di autenticazione dalla casella **itsupport** (funzione `send-auth-email`).	Entra admin center → App registrations; Exchange Online (permesso limitato alla casella)
Exchange Online	Casella da cui partono le email di login/registrazione/password (itsupport).	Exchange admin center
Resend	Servizio SMTP usato finora da Supabase per le email di autenticazione. **Da dismettere** quando l'hook `send-auth-email` è attivo e funzionante.	resend.com + Supabase → Authentication → Emails → SMTP
Google Fonts	Caratteri tipografici del sito (Plus Jakarta Sans, Fraunces).	Nessuna gestione necessaria
esm.sh / npm	Fonti delle librerie usate dalle Edge Functions al momento del deploy.	Nessuna gestione necessaria
Asset Manager H-FARM	Inventario dei dispositivi; le etichette QR contengono il link della scheda (es. `.../hardware/25142/view`), la piattaforma ne ricava l'ID.	assetmanager.h-farm.com
SharePoint / OneDrive	Registri Excel (onboarding, offboarding, camere) e foto.	Microsoft 365

Servizio	A cosa serve	Dove si gestisce
Cisco Meraki Vision	Telecamere (solo link).	vision.meraki.com

Librerie principali del codice (nel file `package.json`)

Libreria	Uso
Next.js 16 + **React 19** + **TypeScript**	Il framework con cui è costruito il sito.
@supabase/supabase-js	Collegamento a Supabase (login, dati, funzioni).
jsqr	Lettura dei QR code dalla fotocamera (studenti ed etichette dispositivi).
qrcode	Generazione delle card QR degli studenti.
write-excel-file	Esportazioni in Excel (storico, budget, ecc.).

5. La struttura dei file

```
management-platform/
├── package.json            ← elenco librerie e comandi (dev, build, lint)
├── next.config.ts          ← configurazione Next.js: "output: export" = sito statico
├── .env.example            ← modello delle variabili d'ambiente (senza valori reali)
├── README.md               ← vecchio readme (in parte superato: vale questa guida)
├── AGENTS.md / CLAUDE.md   ← note per gli assistenti di programmazione
├── public/                 ← file pubblicati così come sono
│   ├── _headers            ← regole di sicurezza HTTP per Cloudflare (CSP ecc.)
│   ├── robots.txt          ← vieta l'indicizzazione sui motori di ricerca
│   ├── site.webmanifest    ← dati per "Aggiungi a schermata Home" (web app)
│   └── logo…, his-mp-…     ← logo e icone
├── src/                    ← CODICE DEL SITO (frontend)
│   ├── app/                ← una cartella = una pagina (vedi tabella sotto)
│   │   ├── layout.tsx      ← struttura comune a tutte le pagine (titolo, icone)
│   │   ├── page.tsx        ← la home (dashboard o pagina di accesso)
│   │   ├── globals.css     ← stile generale
│   │   ├── shell.css       ← stile della "cornice" (menu laterale, barra in alto, dashboard)
│   │   ├── forms.css       ← stile dei moduli
│   │   ├── budget.css      ← stile pagine budget e storico
│   │   └── tasks.css       ← stile Task Manager
│   ├── components/         ← "mattoncini" riutilizzati in più pagine
│   └── lib/                ← logica: collegamento a Supabase, permessi, dati
└── supabase/               ← CODICE DEL BACKEND (si installa su Supabase a mano)
    ├── migrations/         ← 22 file SQL che costruiscono il database
    ├── functions/          ← le 3 Edge Functions
    ├── notifications/      ← istruzioni e modelli per la notifica Teams
    └── checks/             ← security_check.sql: controlli di sicurezza del DB
```

> **Attenzione:** le cartelle `node_modules/`, `.next/` e `out/` sono generate automaticamente e **non** vanno modificate né caricate su GitHub (sono escluse da `.gitignore`).

5.1 Le pagine (`src/app/`)

Ogni cartella dentro `src/app/` corrisponde a un indirizzo del sito. Esempio: `src/app/device-history/` → `https://<sito>/device-history`.

Indirizzo	Cosa contiene	Chi la vede (permesso)
`/`	Dashboard (se loggato) o pagina di benvenuto/accesso	Tutti
`/login`, `/signup`, `/reset-password`	Accesso, registrazione, nuova password	Tutti (non loggati)
`/mfa`	Inserimento / attivazione del codice MFA	Utenti loggati
`/guest`	Pagina d'attesa per chi non ha ancora un ruolo	Guest
`/profile`	Il mio profilo: dati, foto, email, password, MFA, aspetto	Tutti i loggati
`/it`	Hub della sezione IT	`it`
`/student-checkinout-hub`, `/modulo-student`	Consegne/restituzioni studenti (modulo)	`it.checkin_student`
`/employee-checkinout-hub`, `/modulo-employee`	Consegne/restituzioni dipendenti (modulo)	`it.checkin_employee`
`/qr`	Generatore card QR studente	`it.checkin_student`
`/device-history`	Storico assegnazioni (ricerca, modifica, elimina, export)	`it.history`
`/it-registries-hub`	Registri risposte	`it.registries`
`/budget-management/venezia` · `/vicenza` · `/rosa`	Budget delle tre scuole	Solo Owner/Super Admin/Admin
`/budget-management`	Panoramica scuole	Solo Owner/Super Admin/Admin
`/task-manager`	Task Manager	Solo Owner/Super Admin/Admin
`/hr`, `/employee-management-hub`, `/hr-registry-hub`	Hub HR e registri	`hr`, `hr.registries`
`/onboarding`	Modulo nuovo dipendente	`hr.onboarding`
`/employee-management`	Modulo offboarding	`hr.offboarding`
`/boarding`, `/room-assignment-hub`, `/room-assignment`	Boarding e assegnazione camere	`boarding`, `boarding.rooms`
`/student-office`	Student Office	`office`
`/backend`	Hub Gestione Backend	Owner, Super Admin
`/user-management`	Utenti e ruoli	Owner, Super Admin
`/role-permissions`	Matrice permessi per ruolo	Owner, Super Admin
`/audit-log`	Registro attività	Owner, Super Admin

Indirizzo	Cosa contiene	Chi la vede (permesso)
`/security-settings`	MFA per ruolo, timeout, dispositivi ricordati	Owner, Super Admin

Di solito ogni pagina ha due file: `page.tsx` (la "busta": titolo e controllo accessi) e `NomeClient.tsx` (il contenuto vero e proprio).

5.2 I componenti (`src/components/`)

File	Cosa fa
`AppShell.tsx`	La cornice: menu laterale, barra in alto, selettore società, menu mobile, zoom automatico per schermi grandi.
`AuthGuard.tsx`	Il "buttafuori": controlla login, MFA e permessi prima di mostrare una pagina.
`Dashboard.tsx`	La home dopo il login, diversa per società e ruolo; selettore società dopo il login.
`DeviceCheckoutForm.tsx`	Il modulo consegna/restituzione (studenti e dipendenti) con validazione di tutti i campi.
`AssetIdField.tsx`	Campo Asset ID con scansione dell'etichetta QR del dispositivo.
`QrScanner.tsx`	Scanner del QR dello studente (precompila i dati).
`SignaturePad.tsx`	Riquadro per la firma (richiede un tratto vero, non un tocco).
`HubPage.tsx`	Pagine "hub" a schede (IT, HR, Boarding…).
`IdleWatcher.tsx`	Disconnette dopo il tempo di inattività previsto per il ruolo.
`Turnstile.tsx`	Il controllo anti-bot di Cloudflare nelle pagine di accesso.
`Topbar` , `Header` , `Footer` , `Icon` , `useToast`	Elementi grafici e messaggi a comparsa.

5.3 La logica (`src/lib/`)

File	Cosa fa
`supabase.ts`	Crea il collegamento a Supabase usando le variabili d'ambiente.
`auth.ts`	Login, registrazione, recupero password, lettura profilo, controllo accessi, invio moduli a `submit-form` .
`mfa.ts` , `trustedDevice.ts`	MFA e "ricorda questo dispositivo".
`idle.ts`	Timeout per inattività.
`nav.ts`	Il menu e la regola "chi vede cosa" (incluse le sezioni riservate agli admin).
`permissions.ts`	Elenco dei permessi e lettura/modifica della matrice.
`companies.ts`	Le tre società e quale società vede ogni ruolo.
`hubs.tsx`	Contenuto delle pagine hub (schede e link).
`deviceLog.ts` , `deviceExport.ts` , `deviceIcons.tsx`	Storico dispositivi: salvataggio, lettura, modifica, export Excel.
`powerAutomate.ts`	Invio dei moduli dispositivi a `notify-powerautomate` .

File	Cosa fa
`qrCamera.ts`	Fotocamera e lettura QR (alta risoluzione, lettore nativo del telefono se disponibile).
`image.ts`	Compressione delle foto prima dell'invio.
`budgets.ts`, `dashboard.ts`	Commesse, spese, dati della dashboard.
`tasks.ts`	Task Manager.
`users.ts`, `profile.ts`	Gestione utenti e profilo personale.
`excel.ts`	Esportazione Excel generica.
`uiPrefs.ts`	Preferenze di aspetto (zoom, densità).
`features.ts`	Interruttori: `SIGNUP_ENABLED` (registrazione pubblica attiva/disattiva), `PRIVACY_POLICY_URL`.

6. Come funziona il frontend

6.1 Sito statico

`next.config.ts` contiene `output: 'export'`: la build produce una cartella `out/` con **solo file HTML, CSS e JavaScript**. Cloudflare li pubblica così come sono. Non esiste un server della piattaforma: ogni dato arriva da Supabase, direttamente dal browser dell'utente, con il suo token di accesso.

6.2 Variabili d'ambiente del sito

Sono valori inseriti al momento della build (in **Cloudflare Pages → Settings → Environment variables**, e in locale nel file `.env.local`, che non va su GitHub):

Variabile	Valore	È segreta?
`NEXT_PUBLIC_SUPABASE_URL`	`https://gnahighyfoirnadhihez.supabase.co`	No
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Chiave *anon public* di Supabase (Project Settings → API)	No (è pubblica per natura)
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`	*Site key* del widget Turnstile	No

Tutto ciò che inizia con `NEXT_PUBLIC_` finisce nel browser: **non metterci mai segreti**.

6.3 Intestazioni di sicurezza (`public/_headers`)

Cloudflare aggiunge a ogni pagina regole di sicurezza:

- **Content-Security-Policy (CSP):** il sito può caricare script solo da sé stesso e da Cloudflare Turnstile, caratteri da Google Fonts e può collegarsi **solo** a `gnahighyfoirnadhihez.supabase.co` e Turnstile. *Se un giorno cambi progetto Supabase, devi aggiornare questo indirizzo qui, altrimenti il sito non riuscirà a collegarsi.*
- **X-Frame-Options: DENY:** il sito non può essere incorporato in altre pagine.
- **Permissions-Policy:** consente solo la fotocamera (per QR e foto); microfono e posizione bloccati.
- **HSTS:** obbliga l'uso di HTTPS.
- **X-Robots-Tag / robots.txt:** il sito non viene indicizzato da Google.

6.4 Dati salvati nel browser

Alcune preferenze restano nel browser dell'utente (localStorage). Non contengono dati sensibili:

Chiave	Contenuto
`sb-…-auth-token`	La sessione di Supabase (token di accesso).
`his:lastActivity`, `his:idleMinutes`	Ultima attività e minuti di timeout del ruolo.
`his:mfaDevice`, `his:mfaRememberChoice`	Token casuale del "ricorda questo dispositivo" (nel database c'è solo la sua impronta cifrata).
`his:company`, `his:companyPick`	Società selezionata; richiesta di scelta dopo il login.
`his:ui`	Preferenze di aspetto.
`his:postLoginRedirect`	Pagina da riaprire dopo il login.

7. Ruoli, permessi e società

7.1 I ruoli

Ruolo	Descrizione
owner	Il livello più alto. Fa tutto ciò che fa un Super Admin e in più: solo un Owner può dare/togliere il ruolo Owner, modificare o eliminare un altro Owner, azzerarne l'MFA, cambiare le regole MFA del ruolo Owner. Esiste sempre almeno un Owner.
superadmin	Controllo completo della piattaforma, inclusa l'Amministrazione (utenti, permessi, registro, sicurezza). Non può toccare gli Owner.
admin	Vede tutte le sezioni operative di tutte le società, inclusi Budget e Task Manager. Non vede l'Amministrazione.
it	Sezione IT (consegne, storico, registri) — H-IS Venezia.
hr	Sezione HR — H-IS Venezia.
boarding	Sezione Boarding — H-IS Venezia.
office	Student Office — H-IS Venezia.
parent	Ruolo storico, oggi senza sezioni.
office.hvi, **teachers.hvi**	Personale di H-IS Vicenza (al momento senza sezioni assegnate).
office.hro, **teachers.hro**	Personale di H-IS Rosà (al momento senza sezioni assegnate).
guest	Ruolo di chi si è appena registrato: non vede nulla finché un amministratore non assegna un ruolo.

I ruoli si assegnano da **Amministrazione → Utenti e ruoli**. Il ruolo è salvato nella tabella `profiles` (colonna `role`).

7.2 I permessi

Ogni ruolo ha un insieme di **permessi** (cosa può aprire). Si gestiscono da **Amministrazione → Permessi ruoli** (matrice ruoli × permessi). Owner e Super Admin hanno sempre tutto; Guest non ha niente.

Permesso	Cosa apre
`it`	Sezione IT (pagina e menu)
`it.checkin_student`	Moduli consegna/restituzione studenti, card QR
`it.checkin_employee`	Moduli consegna/restituzione dipendenti
`it.history`	Storico assegnazioni: lettura, ricerca, modifica, eliminazione, export
`it.registries`	Registri risposte IT
`it.budget` ADMIN	Budget H-IS Venezia
`vi.budget` ADMIN	Budget H-IS Vicenza
`ro.budget` ADMIN	Budget H-IS Rosà
`it.tasks` ADMIN	Task Manager
`hr`, `hr.onboarding`, `hr.offboarding`, `hr.registries`	Sezione HR e sue funzioni
`boarding`, `boarding.rooms`	Boarding e assegnazione camere
`office`	Student Office

ADMIN = **riservato a Owner, Super Admin e Admin** (migrazione 0022). Questi permessi **non si possono concedere** ad altri ruoli nemmeno dalla matrice: il database rifiuta.

I permessi sono applicati **in due punti**: nelle pagine (menu e accesso) e **nel database** (regole RLS e funzioni). Quindi anche chi provasse a interrogare Supabase direttamente non otterrebbe dati non permessi.

7.3 Le tre società

Società	Chi la vede
H-IS Venezia	Ruoli storici (it, hr, boarding, office, parent) + Owner/Super Admin/Admin
H-IS Vicenza	office.hvi, teachers.hvi + Owner/Super Admin/Admin
H-IS Rosà	office.hro, teachers.hro + Owner/Super Admin/Admin

Chi può vedere più società (Owner, Super Admin, Admin) **sceglie la società dopo il login** e può cambiarla dal menu laterale. Menu e dashboard mostrano solo le sezioni della società scelta. La società è un modo di organizzare la piattaforma; la protezione dei dati è data dai permessi.

7.4 Sicurezza dell'accesso

Funzione	Come funziona	Dove si imposta
Password	Gestita da Supabase Auth (mai salvata in chiaro).	—
Captcha Turnstile	Su login, registrazione, recupero password.	Cloudflare + Supabase Auth

Funzione	Come funziona	Dove si imposta
MFA per ruolo	Per ogni ruolo si decide se l'app di autenticazione è obbligatoria. Applicata anche nel database: senza secondo fattore i dati protetti restano chiusi.	Amministrazione → Sicurezza
Ricorda questo dispositivo	Dopo il codice MFA il browser può essere "fidato" per N ore (default 24; 0 = disattivato). La password resta sempre obbligatoria.	Amministrazione → Sicurezza
Timeout per inattività	Disconnessione automatica dopo N minuti senza attività (5–1440, default 60), diverso per ruolo.	Amministrazione → Sicurezza
Registro attività	Ogni cambio di ruolo, utente, permesso, MFA, record dello storico, budget viene registrato.	Amministrazione → Registro attività
Registrazione pubblica	Attiva/disattiva con `SIGNUP_ENABLED` in `src/lib/features.ts` (va disattivata anche in Supabase → Sign In / Providers). I nuovi utenti nascono **guest**.	Codice + Supabase

8. Il backend: Supabase

Supabase raccoglie quattro servizi:

1. **Auth** — gestisce gli account (email+password), la conferma email, il recupero password, l'MFA, le sessioni. - *Authentication → URL Configuration*: indirizzo del sito e indirizzi di ritorno ammessi (`/login`, `/reset-password`, pagina principale). - *Authentication → Emails*: modelli email e SMTP (Resend). - *Authentication → Auth Hooks*: hook **Send Email** → funzione `send-auth-email` (quando attivato, sostituisce Resend). - *Authentication → Attack Protection*: captcha Turnstile (secret key). - *Authentication → Multi-Factor*: MFA TOTP abilitata.
2. **Database (PostgreSQL)** — tabelle, regole RLS, funzioni e trigger (capitolo 9).
3. **Edge Functions** — i tre programmi lato server (capitolo 10).
4. **Vault** — segreti usati dal database (capitolo 11).

Piano Free — cose da sapere: - 500 MB di database: ampiamente sufficienti (le foto non vanno nel database, le firme sono compresse). - **Nessun backup automatico** sul piano Free: è consigliato un export periodico (vedi capitolo 15). - Il progetto viene **messo in pausa dopo circa una settimana senza utilizzo** (es. vacanze estive): si riattiva dalla dashboard di Supabase, i dati non si perdono.

9. Cosa salva il database

9.1 Le tabelle

Tabella	Cosa contiene	Chi può accedere
`profiles`	Un profilo per utente: nome, cognome, email (copia di quella di login), **ruolo**, foto profilo (piccola, max 150 KB).	Ognuno il proprio; Owner/Super Admin tutti (tramite funzioni)
`auth.users` *(gestita da Supabase)*	Account di accesso: email, password cifrata, fattori MFA.	Solo Supabase

Tabella	Cosa contiene	Chi può accedere
`student_device_log`	**Storico assegnazioni studenti**: data, consegna/restituzione, **email** dello studente (nessun nome), scuola, MacBook ID, iPad ID, caricatori e cavi inclusi (sì/no), chi ha firmato, **firma** (immagine compressa, max 60 KB), chi ha registrato.	Permesso `it.history` (lettura, modifica, eliminazione); inserimento con `it.checkin_student`
`budget_lines`	Le **commesse** di ogni scuola: nome, descrizione, colore, stanziamento.	Permesso budget della scuola (solo admin)
`budget_adjustments`	Storico delle modifiche agli stanziamenti (creazione, aumento/diminuzione, rimozione, motivo, chi).	Come sopra
`it_budget_expenses`	Le **spese**: scuola, commessa, descrizione, fornitore, importo, data, note, chi l'ha inserita.	Come sopra
`it_tasks`	Le **task**: titolo, descrizione, note, gruppo, assegnatari, stato, priorità, date, ore stimate.	Permesso `it.tasks` (solo admin)
`it_subtasks`	Le sotto-task.	Come sopra
`it_task_groups`	I gruppi di progetto.	Come sopra
`it_task_members`	Le persone assegnabili alle task (nome, email per Teams, colore). Si gestisce dal Table Editor.	Come sopra
`role_permissions`	La matrice "ruolo → permesso".	Lettura per tutti i loggati; modifica solo Owner/Super Admin tramite funzione
`mfa_role_policy`	Per ogni ruolo: MFA obbligatoria sì/no, minuti di timeout.	Modifica solo Owner/Super Admin
`platform_settings`	Impostazioni generali (es. ore del "ricorda dispositivo").	Modifica solo Owner/Super Admin
`mfa_trusted_devices`	Dispositivi ricordati: solo l'**impronta cifrata** (SHA-256) del token, scadenza.	Solo funzioni del database
`audit_log`	**Registro attività**: chi, quando, cosa (ruoli, utenti, permessi, MFA, storico, budget). Non modificabile dalla piattaforma.	Solo Owner/Super Admin in lettura

9.2 Cosa **non** viene salvato nel database

- **Nomi e cognomi degli studenti**: per scelta, lo storico conserva solo l'email.
- **Foto dei dispositivi e dei danni**: vanno solo a Power Automate (salvate su OneDrive/SharePoint dal flow).
- **Moduli onboarding, offboarding, assegnazione camere, consegne dipendenti**: non sono salvati in Supabase; vengono inoltrati a Power Automate, che li registra su Excel/SharePoint.
- **Password**: gestite da Supabase in forma cifrata, mai leggibili.

9.3 Funzioni e trigger principali

Le operazioni delicate passano da **funzioni nel database** che verificano i permessi. Le più importanti:

Funzione	A cosa serve
`has_permission`, `my_permissions`, `my_role`	Dicono cosa può fare l'utente collegato (inclusa la regola MFA).

Funzione	A cosa serve
`admin_list_users`, `admin_update_user`, `admin_delete_user`, `admin_reset_mfa`	Gestione utenti (Owner/Super Admin, con protezione degli Owner).
`admin_set_role_permission`	Modifica la matrice permessi (rifiuta i permessi riservati agli admin).
`admin_set_mfa_policy`, `admin_set_idle_timeout`, `admin_set_mfa_remember_hours`	Impostazioni di sicurezza.
`mfa_ok`, `mfa_trust_device`, `mfa_forget_my_devices`	MFA e dispositivi ricordati.
`device_log_update`	Modifica di un record dello storico (con registrazione prima/dopo nel registro attività).
`budget_create_line`, `budget_adjust_line`, `budget_archive_line`	Commesse e stanziamenti.
`purge_device_log_signatures(mesi)`	Cancella le firme più vecchie di N mesi (conservazione GDPR).
`audit_write`	Scrive nel registro attività (usata dai trigger).

Trigger (azioni automatiche):

Trigger	Quando scatta	Cosa fa
`profiles_role_guard`	Creazione/modifica profilo	Forza `guest` sui nuovi utenti, impedisce cambi di ruolo non autorizzati, protegge gli Owner.
`profiles_audit`, `student_device_log_audit`	Modifiche/eliminazioni	Scrivono nel registro attività.
`on_auth_user_email_changed`	Cambio email confermato	Copia la nuova email nel profilo.
`it_tasks_notify_assignees`	Nuova task o nuovi assegnatari	Chiama il flow Power Automate → messaggio Teams ai nuovi assegnatari (via `pg_net`).
`it_tasks_sync_assignees`, `it_tasks_completed`, `*_touch`	Modifiche alle task	Tengono allineati assegnatari, data di completamento, data di modifica.

10. Le Edge Functions

Si trovano in **Supabase → Edge Functions** e il loro codice è nel repository in `supabase/functions/`. Per aggiornarne una: si apre la funzione nel dashboard, si incolla il nuovo `index.ts` e si fa **Deploy** (oppure, da terminale, `supabase functions deploy <nome>`).

10.1 `submit-form`

- **Cosa fa:** riceve un modulo dal sito, verifica che l'utente sia loggato, abbia il permesso giusto e (se richiesto) abbia superato l'MFA, poi inoltra i dati al flow Power Automate corrispondente. Gli URL dei flow sono nei secret: **non arrivano mai al browser**.
- **Chi la chiama:** `src/lib/auth.ts` → `submitForm()`.
- **Moduli gestiti:**

Tipo modulo (`form_type`)	Permesso richiesto	Secret con l'URL del flow
`onboarding`	`hr.onboarding`	`WEBHOOK_ONBOARDING`
`offboarding`	`hr.offboarding`	`WEBHOOK_OFFBOARDING`
`student_checkin`	`it.checkin_student`	`WEBHOOK_STUDENT_CHECKIN`
`student_checkout`	`it.checkin_student`	`WEBHOOK_STUDENT_CHECKOUT`
`employee_checkin`	`it.checkin_employee`	`WEBHOOK_EMPLOYEE_CHECKIN`
`employee_checkout`	`it.checkin_employee`	`WEBHOOK_EMPLOYEE_CHECKOUT`
`room`	`boarding.rooms`	`WEBHOOK_ROOM`

- **Limiti:** richiesta massima 16 MB (le foto sono già compresse nel browser).
- **Secret opzionali:** `ALLOWED_ORIGINS` (domini autorizzati), `TURNSTILE_SECRET` (**non impostarlo**: i moduli non inviano ancora il captcha e verrebbero rifiutati).

10.2 `notify-powerautomate`

- **Cosa fa:** dopo ogni consegna/restituzione dispositivo, invia al flow Power Automate un riepilogo completo (dati, dispositivi, ID, caricatori e cavi, firma come immagine, foto) per Excel, OneDrive e l'email di riepilogo. Accetta solo utenti con permesso `it.checkin_student` / `it.checkin_employee` e MFA superata se richiesta.
- **Chi la chiama:** `src/lib/powerAutomate.ts`.
- **Secret:** `POWER_AUTOMATE_WEBHOOK_URL`; opzionale `ALLOWED_ORIGINS`.
- **Limite:** 30 MB per richiesta.
- **Documentazione del flow:** `supabase/functions/notify-powerautomate/README.md` (campi disponibili), `sample-payload.json`, `request-schema.json`, `email-recap.html` (modello email).

> Una consegna studente quindi genera **tre azioni**: (1) `submit-form` → flow `WEBHOOK_STUDENT_CHECKOUT`; (2) una riga in `student_device_log`; (3) `notify-powerautomate` → flow di riepilogo. Se (2) o (3) falliscono l'utente vede un avviso, ma il modulo risulta comunque inviato.

10.3 `send-auth-email`

- **Cosa fa:** è l'**hook "Send Email"** di Supabase Auth. Ogni volta che Supabase deve inviare un'email di autenticazione (conferma registrazione, recupero password, cambio email, invito, magic link, codici), chiama questa funzione, che la spedisce **dalla casella itsupport** tramite **Microsoft Graph**, con la grafica della piattaforma e in italiano. Sostituisce Resend.
- **Chi la chiama:** Supabase Auth (non il sito). La richiesta è firmata: la funzione rifiuta chi non ha la firma corretta.
- **Impostazione obbligatoria: Verify JWT disattivato** (la chiama Supabase, non un utente).
- **Secret:** `SEND_EMAIL_HOOK_SECRET`, `MS_TENANT_ID`, `MS_CLIENT_ID`, `MS_CLIENT_SECRET`, `MAIL_FROM` (es. `itsupport@h-farmschool.com`), opzionale `MAIL_FROM_NAME`.
- **Permessi Microsoft:** app registrata in Entra "HIS Management Platform – Mail" **senza** `Mail.Send` nelle API permissions; il permesso di invio è concesso in Exchange Online con *RBAC for Applications*, **limitato alla sola casella itsupport**.
- **Istruzioni complete:** `supabase/functions/send-auth-email/README.md`.
- **Stato:** pronta; si attiva da *Authentication → Auth Hooks → Send Email hook*. Per tornare a Resend basta disattivare l'hook.

- **Scadenza:** il *client secret* dell'app Entra scade (max 24 mesi): rinnovarlo in tempo e aggiornare `MS_CLIENT_SECRET`.

11. Secret e chiavi: dove sono e a cosa servono

> Questa tabella indica **dove** sono custoditi i valori. I valori veri **non** sono in questa guida né nel codice.

11.1 Nel sito (Cloudflare Pages → Environment variables)

Nome	A cosa serve	Riservato?
`NEXT_PUBLIC_SUPABASE_URL`	Indirizzo del progetto Supabase	No
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Chiave pubblica di Supabase	No
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`	Chiave pubblica del captcha	No

11.2 Supabase → Edge Functions → Secrets

Nome	Usato da	A cosa serve
`WEBHOOK_ONBOARDING`	submit-form	URL del flow onboarding
`WEBHOOK_OFFBOARDING`	submit-form	URL del flow offboarding
`WEBHOOK_STUDENT_CHECKIN`	submit-form	URL del flow restituzione studenti
`WEBHOOK_STUDENT_CHECKOUT`	submit-form	URL del flow consegna studenti
`WEBHOOK_EMPLOYEE_CHECKIN`	submit-form	URL del flow restituzione dipendenti
`WEBHOOK_EMPLOYEE_CHECKOUT`	submit-form	URL del flow consegna dipendenti
`WEBHOOK_ROOM`	submit-form	URL del flow assegnazione camere
`POWER_AUTOMATE_WEBHOOK_URL`	notify-powerautomate	URL del flow di riepilogo dispositivi
`ALLOWED_ORIGINS` *(opzionale)*	submit-form, notify-powerautomate	Domini del sito autorizzati a chiamare le funzioni
`TURNSTILE_SECRET` *(non impostare per ora)*	submit-form	Captcha sui moduli
`SEND_EMAIL_HOOK_SECRET`	send-auth-email	Firma dell'hook email (da Authentication → Auth Hooks)
`MS_TENANT_ID`	send-auth-email	ID del tenant Microsoft
`MS_CLIENT_ID`	send-auth-email	ID dell'app Entra "Mail"
`MS_CLIENT_SECRET`	send-auth-email	Segreto dell'app Entra "Mail" (**scade**)
`MAIL_FROM` / `MAIL_FROM_NAME`	send-auth-email	Casella mittente e nome visualizzato
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`	tutte	**Inseriti automaticamente da Supabase**: non vanno creati.

Secret non più usati (eliminabili se presenti): `WEBHOOK_MEDICINE`, `WEBHOOK_DIET`, `WEBHOOK_PURCHASE_VICENZA`.

11.3 Supabase → Vault (segreti del database)

Nome	A cosa serve
`task_notify_webhook_url`	URL del flow Power Automate che invia la notifica Teams per le task
`platform_url`	Indirizzo della piattaforma (per il link nella card Teams)

11.4 Altri segreti

Dove	Cosa
Supabase → Project Settings → API	**Service role key** (segretissima: mai nel sito) e anon key
Supabase → Authentication → Attack Protection	Secret key di Cloudflare Turnstile
Supabase → Authentication → Emails → SMTP	Credenziali SMTP di Resend (finché in uso)
Microsoft Entra → App registrations → "HIS Management Platform – Mail"	Client secret (copiato in `MS_CLIENT_SECRET`)
Power Automate	Gli URL dei trigger HTTP dei flow (chi li conosce può inviare dati al flow: trattarli come password)
GitHub / Cloudflare / Supabase	Gli account di accesso alle tre console (usare account di team, con MFA)

Se un segreto finisce nelle mani sbagliate: rigeneralo nel servizio d'origine (nuovo URL del flow, nuovo client secret, nuova chiave), aggiorna il valore nel secret corrispondente e, per le Edge Functions, non serve rifare il deploy (i secret si leggono a ogni chiamata).

12. Come funzionano i flussi principali

12.1 Login

1. L'utente apre il sito e inserisce email e password; Turnstile verifica che non sia un bot.
2. Supabase Auth controlla le credenziali e restituisce un token.
3. Se il ruolo richiede MFA, la pagina `/mfa` chiede il codice dell'app (a meno che il dispositivo sia "ricordato").
4. La piattaforma legge profilo, ruolo e permessi; chi vede più società sceglie la società.
5. Dopo N minuti di inattività l'utente viene disconnesso.

12.2 Consegna o restituzione di un dispositivo (studente)

1. IT apre *Consegne studenti* → Check-out (consegna) o Check-in (restituzione).
2. (Facoltativo) scansiona la card QR dello studente: i dati si compilano da soli.
3. Seleziona i dispositivi; per MacBook/iPad/Accessories inserisce o scansiona l'**Asset ID** dall'etichetta (l'URL dell'Asset Manager, es. `.../hardware/25142/view` → `25142`).
4. Per MacBook e iPad aggiunge almeno 2 foto (fronte e retro) ed eventuali foto dei danni.
5. Sceglie chi firma e raccoglie la firma. **Tutti i campi sono obbligatori** (tranne la seconda email del genitore).
6. All'invio: `submit-form` → flow; riga nello **storico** (`student_device_log`); `notify-powerautomate` → Excel, OneDrive, email di riepilogo.

12.3 Storico assegnazioni

Ricerca per email o ID, filtri, stato attuale dei dispositivi ("chi ha cosa"), badge caricatore/cavo, **matita** per correggere un record, **cestino** per eliminarlo, export Excel. Ogni modifica/eliminazione va nel registro attività.

12.4 Budget

Per ogni scuola: commesse con stanziamento, spese, residuo. Pulsante *Nuova commessa*; pulsante *Budget* su ogni commessa per aggiungere/togliere stanziamento con motivo; una commessa senza spese si può rimuovere. Tutto tracciato in `budget_adjustments` e nel registro attività.

12.5 Task Manager e Teams

Quando una task viene creata o le si aggiungono assegnatari, il database (trigger + `pg_net`) chiama il flow Power Automate, che invia una card Teams solo ai **nuovi** assegnatari (non a chi fa la modifica). Serve l'email in `it_task_members`.

12.6 Email di autenticazione

Registrazione, recupero password, cambio email: Supabase prepara il link e invia l'email tramite **Resend (SMTP)** oppure, con l'hook attivo, tramite `send-auth-email` → **Microsoft Graph → casella itsupport**.

13. Le migrazioni del database

Le migrazioni si trovano in `supabase/migrations/` e vanno eseguite **una volta, in ordine**, nel **SQL Editor** di Supabase (si apre il file, si copia tutto, si incolla, *Run*). Sono **rieseguibili**: rilanciarne una non rompe nulla, ma alcune note in testa ai file indicano quali migrazioni rilanciare dopo.

N.	Contenuto
0001	Tabella spese budget IT
0002	Spese divise per scuola
0003	Task Manager (task, sotto-task, gruppi, membri)
0004	Note sulle task
0005	Storico dispositivi studenti (`student_device_log`)
0006	Eliminazione record dello storico
0007	Nuovi utenti come `guest`; solo Super Admin cambia ruoli
0008	Ruolo Super Admin e Gestione Utenti
0009	Nome e cognome dalla registrazione al profilo
0010	Correzione cambio ruoli da Super Admin
0011	Più assegnatari per task
0012	Foto profilo e cambio email
0013	Notifica Teams sull'assegnazione delle task (Vault + pg_net)
0014	Sicurezza e GDPR: registro attività, email protetta, pulizia firme
0015	MFA obbligatoria per ruolo

N.	Contenuto
0016	Timeout per ruolo, "ricorda dispositivo", sezioni admin
0017	Ruolo Owner e permessi per ruolo
0018	Tre società, nuovi ruoli, commesse nel database
0019	Rimozione della sezione Purchases
0020	Modifica dei record dello storico
0021	Caricatori e cavi nello storico
0022	Budget e Task Manager solo per Owner, Super Admin, Admin

Ricostruire il database da zero (es. nuovo progetto Supabase): eseguire 0001 → 0022 in ordine, poi creare il primo Owner (vedi 15.1), inserire i segreti del Vault (0013) e ricreare i secret delle Edge Functions.

Il file `supabase/checks/security_check.sql` contiene query di verifica della sicurezza (RLS attiva, policy presenti) da eseguire nel SQL Editor.

14. Come pubblicare una modifica

14.1 Modifica al sito (frontend)

1. Modifica i file nella copia locale `~/Downloads/management-platform/management-platform`.
2. (Facoltativo) prova in locale dal Terminale: `npm install (solo la prima volta o se cambiano le librerie) npm run dev (sito di prova su http://localhost:3000)`
3. Pubblica: `cd ~/Downloads/management-platform/management-platform git add -A git commit -m "Descrizione della modifica" git push`
4. Cloudflare Pages vede il push, esegue la build (`npm run build` , che genera la cartella `out/`) e pubblica. In pochi minuti la nuova versione è online. In **Cloudflare → progetto → Deployments** si vede l'esito; in caso di errore si legge il log e la versione precedente resta online.

14.2 Modifica al database

1. Crea (o ricevi) un nuovo file di migrazione numerato, es. `0023_….sql` , in `supabase/migrations/` .
2. Eseguilo nel **SQL Editor** di Supabase.
3. Fai anche il `git push` , così il file resta nel repository.

> Ordine consigliato quando una modifica tocca sia database sia sito: **prima la migrazione, poi il push**.

14.3 Modifica a una Edge Function

1. Aggiorna il file `supabase/functions/<nome>/index.ts` .
2. Supabase → Edge Functions → → incolla il codice → **Deploy** (rispettando l'impostazione *Verify JWT* indicata nel capitolo 10).
3. `git push` per tenere il codice allineato.

14.4 Modifica a un flow Power Automate

Si fa direttamente in Power Automate. Se cambia l'**URL** del trigger, aggiornare il secret corrispondente (capitolo 11).

15. Manutenzione e operazioni frequenti

15.1 Utenti e ruoli

- **Nuovo dipendente:** si registra da `/signup` (nasce *guest*) → un Owner/Super Admin gli assegna il ruolo da *Utenti e ruoli*.
- **Primo Owner** (solo una volta, nel SQL Editor): `sql update public.profiles set role = 'owner' where id = (select id from auth.users where email = 'nome.cognome@h-farmschool.com');`
- **Telefono MFA perso** da un utente: *Utenti e ruoli* → *Azzera MFA*.
- **Telefono MFA perso dall'unico Owner** (SQL Editor): `sql delete from auth.mfa_factors where user_id = (select id from auth.users where email = 'nome.cognome@h-farmschool.com');`
- **Uscita di un dipendente:** impostare il ruolo a *guest* o eliminare l'utente da *Utenti e ruoli*.

15.2 Membri del Task Manager

Supabase → Table Editor → `it_task_members`: aggiungere persone, email (per Teams), disattivare con `active = false`.

15.3 Conservazione dei dati (GDPR)

- Firme più vecchie di N mesi: `select public.purge_device_log_signatures(24);` (esempio: 24 mesi). Si può pianificare con pg_cron (istruzioni in fondo alla migrazione 0014).
- Il registro attività conserva la storia delle modifiche: non va modificato.

15.4 Backup (piano Free)

Il piano Free **non fa backup automatici** (i backup giornalieri sono inclusi solo nei piani a pagamento). Consigliato almeno una volta al mese:

- esportare le tabelle principali dal **Table Editor** (*Export → CSV*): `profiles`, `student_device_log`, `budget_lines`, `budget_adjustments`, `it_budget_expenses`, `it_tasks`, `it_subtasks`, `it_task_groups`, `it_task_members`, `role_permissions`, `mfa_role_policy`, `audit_log`;
- oppure, per chi usa il Terminale, un backup completo con `pg_dump` usando la stringa di connessione (Project Settings → Database).

Conservare i file in un luogo sicuro e ad accesso ristretto (contengono email e firme), ad esempio una cartella SharePoint del team IT.

15.5 Scadenze da ricordare

Cosa	Quando
Client secret dell'app Entra "Mail"	Alla scadenza scelta (max 24 mesi)
Pausa del progetto Supabase Free	Dopo ~7 giorni senza utilizzo (vacanze)
Rinnovo dominio / account dei servizi	Secondo i contratti

16. Risoluzione dei problemi

Problema	Cosa controllare
Il sito non si apre o mostra una versione vecchia	Cloudflare → Deployments: l'ultima build è andata a buon fine? Svuotare la cache del browser.
"Supabase non raggiungibile" / pagine vuote	Il progetto Supabase è in pausa? (dashboard Supabase → *Restore*). Le variabili `NEXT_PUBLIC_SUPABASE_*` su Cloudflare sono corrette?
Un utente non vede una sezione	Ruolo in *Utenti e ruoli*; permessi in *Permessi ruoli*; società selezionata; MFA superata se richiesta. Budget e Task Manager sono solo per admin.
"Prima attiva la tua MFA"	L'utente deve configurare l'app di autenticazione dal Profilo.
Un modulo non viene inviato	Supabase → Edge Functions → *submit-form* → **Logs**. Secret `WEBHOOK_*` presente? Il flow Power Automate è attivo?
Manca l'email di riepilogo dispositivi	Log di *notify-powerautomate*; cronologia del flow in Power Automate.
Non arriva l'email di registrazione/password	Se l'hook è attivo: log di *send-auth-email* (errori Microsoft 403 = permesso Exchange non ancora attivo, attendere fino a 2 ore). Se l'hook è spento: impostazioni SMTP/Resend. Controllare anche lo spam.
La notifica Teams delle task non arriva	Email della persona in `it_task_members` ; segreti nel Vault; nel SQL Editor `select * from net._http_response order by created desc limit 5;`
Lo scanner QR non legge	Permesso fotocamera del browser; etichetta dentro il quadrato a 10–15 cm; usare la torcia. Se compare "QR non riconosciuto" il messaggio mostra il testo letto.
Errore dopo aver rieseguito una vecchia migrazione	Leggere le note *NOTE: … re-run …* in testa al file e rieseguire le migrazioni indicate (in genere 0016, 0017, 0018, 0021, 0022).

17. Note finali

- Il file `README.md` del repository descrive una fase precedente del progetto (es. Netlify, moduli medicine e dieta poi rimossi): **fa fede questa guida**.
- Ogni modifica importante della piattaforma dovrebbe essere accompagnata da un aggiornamento di questa guida.
- Contatto tecnico di riferimento: team IT di H-FARM International School.